Guía de Despliegue, CI/CD y Ambiente de Staging

Este documento detalla los procedimientos de integración continua, despliegue y operación del ambiente de Staging y Producción para Bull Dog Solution.

1. Pipeline de Integración Continua (CI)

El archivo `.github/workflows/ci.yml` define el flujo de validación automática en cada `push` o `pull_request`:

1. **Instalación de dependencias:** Uso de `pnpm install --frozen-lockfile` con almacenamiento en caché.
2. **Validación Prisma:** Ejecución de `pnpm prisma:validate` y generación del cliente Prisma.
3. **Análisis Estático (Linting):** Ejecución de `pnpm lint` aplicando las reglas de ESLint de arquitectura modular.
4. **Comprobación de Tipos (Typecheck):** Ejecución de `pnpm typecheck` en todos los paquetes del monorepo.
5. **Pruebas Unitarias y de Integración:** Ejecución de `pnpm test` con Vitest.
6. **Validación de OpenAPI:** Ejecución de `pnpm docs:validate` y verificación de contratos OpenAPI contra endpoints.
7. **Compilación (Build):** Generación de artefactos optimizados de backend y frontend web.
8. **Auditoría de Dependencias:** Ejecución de `pnpm audit` verificando 0 vulnerabilidades conocidas.
9. **Pruebas E2E:** Ejecución de pruebas con Playwright en entornos de escritorio y móvil (`pnpm test:e2e`).

2. Configuración de Entornos y Secretos Separados

Se deben mantener tres ambientes completamente aislados:

- **Desarrollo (Local):** `docker-compose.yml` local con PostgreSQL, MinIO y catálogo simulado de Sheets.
- **Staging (Preproducción):** Base de datos PostgreSQL dedicada, bucket S3 dedicado y documento de Google Sheets exclusivo de pruebas (`GOOGLE_SHEETS_SPREADSHEET_ID_STAGING`).
- **Producción:** Infraestructura productiva con políticas de respaldo continuo y hoja principal de clientes y salidas.

Cada entorno cuenta con sus propias claves criptográficas asimétricas para JWT (`JWT_ACCESS_PRIVATE_KEY` , `JWT_REFRESH_PRIVATE_KEY`), credenciales de base de datos y llaves de acceso a almacenamiento de objetos.

3. Topología de Despliegue Recomendada

- **Frontend & API:** Despliegue bajo el mismo dominio público o mediante reverse proxy (ej. Nginx, Caddy o Cloudflare) mapeando `/api` directamente a la instancia de Express y `/` al frontend estático SPA/PWA. Esto elimina complejidades de cookies de terceros y simplifica CORS.
- **Base de Datos:** PostgreSQL gestionado (ej. Neon, Supabase o AWS RDS PostgreSQL) con migración ejecutada mediante `infra/scripts/migrate.sh` antes de conmutar tráfico.

- **Almacenamiento de Objetos:** AWS S3 o Cloudflare R2 con bloqueo de acceso público activado.

4. Procedimiento de Migración y Despliegue

1. **Ejecución de migraciones:**

   ```
   ./infra/scripts/migrate.sh
   ```

2. **Despliegue de versión:** Inicio de contenedores de API y actualización de artefactos frontend.
3. **Smoke Tests Automáticos:**

   ```
   TARGET_URL="https://staging.ejemplo.com" ./infra/scripts/smoke-test.sh
   ```

   El script verifica automáticamente liveness (`/health/live`), readiness (`/health/ready`), métricas (`/metrics`) y documentación OpenAPI.
4. **Procedimiento de Rollback:** En caso de fallo crítico detectado en smoke tests o métricas, se revierte el contenedor a la versión de imagen previa y se restaura el respaldo de base de datos previo mediante `infra/scripts/restore.sh` .